



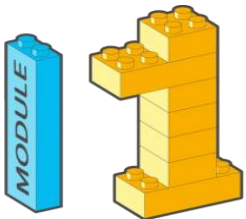
VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_PRINCIPLES OF COMPILER DESIGN



INTRODUCTION TO COMPILATION AND LEXICAL ANALYSIS



Dr. B.V. Baiju,
SCOPE, Assistant Professor
VIT, Vellore

SPECIFICATIONS OF TOKENS

- Specification of tokens **depends on the pattern of the lexeme**.
- Regular expressions is used to specify the different types of patterns that can actually form tokens.

Strings and Languages

Operation on Languages

Regular Expression

Regular Definition

Strings and Languages

1. String

- **Alphabet** or character class is a finite set of symbols. Denoted as Σ

Example :

- The set of digits (symbols) $\Sigma = \{0, 1\}$ forms a binary alphabet.
- ASCII and EBCDIC used in almost every computer, denotes the alphabet A using the set of digits $\{0, 1\}$ i.e. A = 01000001.
- $\Sigma = \{a, b, \dots, z\}$ is the set of lower case letters

- **Symbols** can be **letters**, **digits** and **punctuation**.
- **String** is a **finite set of alphabets**. It is generated from Σ

- $\Sigma = \{a, b\}$ we can derive n number of strings **a, ab,ba,aab,.....**
- $\Sigma = \{0, 1\}$ possible strings are **0,01,1,10,100,010,.....**

2. Language

- Language is a **set or a collection of strings** over some fixed alphabets.

$$L = \{0, 01, 1, 10, 100, 010, \dots\}$$
$$L = \{a, b, ab, ba, abb, \dots\}$$

- $\{\epsilon\}$ – set containing only empty string is language under ϕ .

Operations on strings

Length of String

- Length of the string can be determined by the **number of alphabets in the string**.
- String is represented by the letter 's' and **|s|** represents the length of the string.

$$s = \text{banana}, |s| = 6$$
$$s = 1100, |s| = 4$$

Empty string

- The **empty string** or the string with length 0 is represented by ' **ϵ** '.
- The string does not contain any character

$$|\epsilon| = 0$$

TERM	DEFINITION	EXAMPLE
Prefix of s	A string obtained by removing zero or more trailing symbols of string s	ban is a prefix of banana. S=abcd Prefix: ε, a, ab, abc, abcd
Suffix of s	A string formed by deleting zero or more of the leading symbols of s.	nana is a suffix of banana. s = abcd Suffix: ε, d, cd, bcd, abcd
Substring of s	A string obtained by deleting a prefix and a suffix from s	nan is a substring of banana. s=banana Substring : ε nan,na,anan
Proper prefix, suffix, or substring of s	Any nonempty string x that is a prefix, suffix or substring of s that $s \neq x$.	S= abcd Proper Prefix : a, ab, abc Proper Suffix : d, cd, bcd Substring : bcd, abc, cd, ab
Subsequence of s	Any string formed by deleting zero or more not necessarily contiguous symbols from s	baaa is a subsequence of banana S=abcd Subsequence : abd, bcd, bd

Operation on Languages

- Let **L** be the set $\{A, B, \dots, Z, a, b, \dots, z\}$ and **D** the set $\{0, 1, \dots, 9\}$
- **L** and **D** can be thought in 2 ways
 - L as alphabet consisting of upper and lower case letters
 - D as alphabet consisting of set of ten decimal digits.
- New languages that can be created from L and D by applying the operations are
 1. **$L \cup D$** is the set of letters and digits
 2. **LD** is the set of strings consisting of a letter followed by a digit
 3. **L^4** is the set of all four letter strings
 4. **L^*** is the set of all strings of letters, including epsilon (empty string)
 5. **$L(LUD)^*$** is the set of all strings of letters and digits beginning with a letter
 6. **D^+** is the set of all strings of one or more digits

OPERATION	DEFINITION	EXAMPLE
Union of L and M ($L \cup M$)	$L \cup M = \{ s \mid s \text{ is in } L \text{ or } s \text{ is in } M \}$	If $L = \{a, b\}$ and $M = \{c, d\}$ $L \cup M = \{a, b, c, d\}$
Concatenation of L and M. (LM)	$LM = \{ st \mid s \text{ is in } L \text{ and } t \text{ is in } M \}$	If $L = \{a, b\}$ and $M = \{c, d\}$ $L \cdot M = \{ac, ad, bc, bd\}$
Kleene closure of L (L^*)	L^* denotes “zero or more concatenation of” L	If $L = \{a, b\}$ $L^* = \{\epsilon, a, b, aa, bb, aaa, bbb, \dots\}$
Positive closure of L (L^+)	L^+ denotes “one or more Concatenation of” L	If $L = \{a, b\}$ $L^+ = \{a, b, aa, bb, aaa, bbb, \dots\}$

Example

Let, $L = \{0, 1\}$ and $S = \{a, b, c\}$

Perform $L \cup S$, $L.S$, L^* , L^+

1. Union ($L \cup S$) = $\{0, 1, a, b, c\}$
2. Concatenation ($L.S$) = $\{0a, 0b, 0c, 1a, 1b, 1c\}$
3. Kleene closure (L^*) = $\{\epsilon, 0, 1, 00, \dots\}$
4. Positive closure (L^+) = $\{0, 1, 00, \dots\}$

Regular Expression

- In pascal, an identifier is a letter followed by zero or more letters or digits

$id = letter (letter \mid digit) ^*$

- |** means 'or'
- ()** Paranthese is used to group subexpressions
- *** means zero or more instances of the parenthesised expression

- A regular expression is a **sequence of symbols** used to specify lexeme patterns.
- A regular expression is helpful in **describing the languages** that can be built using operators such as union, concatenation, and closure over the symbols.
- A regular expression **r** that denotes a language **$L(r)$** is built recursively over the smaller regular expression using set of defined rules.
- Regular expressions arise from Chomsky type 3 grammars.
- A Chomsky type 3 grammar has productions of the form:

$A \rightarrow a$ or $A \rightarrow aB$

Must have a single non-terminal on the left-hand side and a right-hand side consisting of a single terminal or single terminal followed by a single non-terminal

Basis

- There are two rules that form the basis:
 - (i) ϵ is a regular expression that denotes $\{\epsilon\}$, (i.e) the set containing the empty string.
 - (ii) If 'a' is a symbol in Σ , then 'a' is a regular expression, and $L(a) = \{a\}$, that is, the language with one string, of length one, with 'a' in its one position.

Induction:

- There are four parts to the induction whereby larger regular expressions are built from smaller ones
- Suppose r and s are regular expressions denoting languages $L(r)$ and $L(s)$, respectively.
 - (i) $(r)|(s)$ is a regular expression denoting the language $L(r) \cup L(s)$.
 - (ii) $(r)(s)$ is a regular expression denoting the language $L(r)L(s)$.
 - (iii) $(r)^*$ is a regular expression denoting $(L(r))^*$.
 - (iv) (r) is a regular expression denoting $L(r)$
- A language denoted by a regular expression is said to be a **regular set**.

- A regular expression is made up of symbols of the language being defined together with operators that support:
 - **concatenation** (traditionally specified by symbol **adjacency** in the regular expression)
 - **alternation** (symbols or groups of symbols are separated by the **|** operator) and
 - **repetition** (symbols or groups of symbols are followed by the ***** operator to signify zero or more repetitions).
- **Parentheses** can also be used to group symbols.
- **Unary operator (*)** has **highest precedence** and is **left associative**
- **Concatenation** has the **second precedence** and is **left associative**
- **Alternation (|)** has the **lowest precedence** and is **left associative**

$(a) \mid ((b))^ (c)$ is équivalent to $a \mid b^* c$*

Both expression denotes set of stings that are either a single a or zero or b's followed by one c

Example

Let $\Sigma = \{a, b\}$

- Regular expression abc denotes the set of strings with the single member $\{abc\}$.
- Regular expression $a|b|c$ denotes the set $\{a, b, c\}$.
- Regular expression $(a|b)(a|b)$ denotes $\{aa, ab, ba, bb\}$
- Regular expression a^* denotes $\{\epsilon, a, aa, aaa, \dots\}$. ϵ is the empty string.
- Regular expression ab^* denotes the infinite set $\{a, ab, abb, abbb, \dots\}$.
- Regular expression $(a|b)^*$ denotes the set of strings made up of **zero or more instances of a or b**.
- Regular expression $a(bc|d)^*e$ denotes the set of strings including $\{ae, abce, abcde, ade, \dots\}$.

- Algebraic laws for regular expressions

AXIOM	DESCRIPTION
$r s = s r$	$ $ is commutative $(3 + 5 = 5 + 3)$
$r (s t) = (r s) t$	$ $ is associative $(2 \times 3) \times 4 = 2 \times (3 \times 4)$
$(rs)t = r(st)$	Concatenation is associative
$r(s t) = rs rt, (s t)r = sr tr$	Concatenation distributes over $ $
$\epsilon r = r\epsilon = r$	ϵ is the identity element for concatenation
$r^* = (r \epsilon)^*$	Relation between $*$ and ϵ
$r^{**} = r^*$	$*$ is idempotent $(0 \times 0 = 0)$

- The **commutative** property states that changing the order of the operands does not change the result of the operation.
- The **associative** property states that the grouping of numbers within an operation does not affect the result.
- The **idempotent** refers to an operation that, when applied multiple times, has the same effect as applying it once

Regular expression examples

1. 0 or 1

Strings: 0, 1

R. E. = $0 \mid 1$

2. 0 or 11 or 111

Strings: 0, 11, 111

R. E. = $0 \mid 11 \mid 111$

3. String having zero or more a .

Strings: ϵ , a , aa , aaa , $aaaa$

R. E. = a^*

4. String having one or more a .

Strings: a , aa , aaa , $aaaa$

R. E. = a^+

5. Regular expression over $\Sigma = \{a, b, c\}$ that represent all string of length 3.

Strings: abc , bca , bbb , cab , aba

R. E. = $(a|b|c)(a|b|c)(a|b|c)$

6. All binary string

Strings: 0, 11, 101, 10101, 1111 ...

R. E. = $(0 \mid 1)^+$

Regular expression examples

7. 0 or more occurrence of either a or b or both

Strings: ϵ , a, aa, abab, bab ...

R.E. = $(a | b)^$*

8. 1 or more occurrence of either a or b or both

Strings: a, aa, abab, bab, bbbaaa ...

R.E. = $(a | b)^+$

9. Binary no. ends with 0

Strings: 0, 10, 100, 1010, 11110 ...

R.E. = $(0 | 1)^ 0$*

10. Binary no. ends with 1

Strings: 1, 101, 1001, 10101, ...

R.E. = $(0 | 1)^ 1$*

11. Binary no. starts and ends with 1

Strings: 11, 101, 1001, 10101, ...

R.E. = $1 (0 | 1)^ 1$*

12. String starts and ends with same character

Strings: 00, 101, aba, baab ...

R.E. = $1 (0 | 1)^ 1$ or $0 (0 | 1)^* 0$
 $a (a | b)^* a$ or $b (a | b)^* b$*

Regular expression examples

13. All string of a and b starting with a

Strings: a, ab, aab, abb...

$$R.E. = a(a \mid b)^*$$

14. String of 0 and 1 ends with 00

Strings: 00, 100, 000, 1000, 1100...

$$R.E. = (0 \mid 1)^*00$$

15. String ends with abb

Strings: abb, babbb, ababb...

$$R.E. = (a \mid b)^*abb$$

16. String starts with 1 and ends with 0

Strings: 10, 100, 110, 1000, 1100...

$$R.E. = 1(0 \mid 1)^*0$$

17. All binary string with at least 3 characters and 3rd character should be zero

Strings: 000, 100, 1100, 1001...

$$R.E. = (0 \mid 1)(0 \mid 1)0(0 \mid 1)^*$$

18. Language which consist of exactly two b's over the set $\Sigma = \{a, b\}$

Strings: bb, bab, aabb, abba...

$$R.E. = a^*ba^*ba^*$$

Regular expression examples

24. The language with $\Sigma = \{a, b, c\}$ where a should be multiple of 3

Strings: *aaa, baaa, bacaba, aaaaaa...* **R.E.** $= ((b|c)^* a (b|c)^* a (b|c)^* a (b|c)^*)^*$

25. Even no. of 0

Strings: *00, 0101, 0000, 100100....* **R.E.** $= (1^* 01^* 01^*)^*$

26. String should have odd length

Strings: *0, 010, 110, 000, 10010....* **R.E.** $= (0|1) ((0|1)(0|1))^*$

27. String should have even length

Strings: *00, 0101, 0000, 100100....* **R.E.** $= ((0|1)(0|1))^*$

28. String start with 0 and has odd length

Strings: *0, 010, 010, 000, 00010....* **R.E.** $= (0) ((0|1)(0|1))^*$

30. String start with 1 and has even length

Strings: *10, 1100, 1000, 100100....* **R.E.** $= 1(0|1)((0|1)(0|1))^*$

31. All string begins or ends with 00 or 11

Strings: *00101, 10100, 110, 01011 ...* **R.E.** $= (00|11)(0|1)^* | (0|1)^* (00|11)$

Regular expression examples

31. Language of all string containing both 11 and 00 as substring

Strings: 0011, 1100, 100110, 010011 ...

*R.E. = $((0|1)^*00(0|1)^*11(0|1)^*) \mid ((0|1)^*11(0|1)^*00(0|1)^*)$*

31. String ending with 1 and not contain 00

Strings: 011, 1101, 1011

R.E. = $(1|01)^+$

32. Language of C identifier

Strings: area, i, redious, grade1

R.E. = $(_ + L)(_ + L + D)^$*

where L is Letter & D is digit

Regular Definition

- The regular definition is the **name given to the regular expression**.
- The regular definition (name) of a regular expression is used in the subsequent expressions.
- The regular definition used in an expression appears as if it is a symbol.
- *If Σ is an alphabet of basic symbols, then a regular definition is a sequence of definitions of the form*

$$\begin{array}{l} d_1 \rightarrow r_1 \\ d_2 \rightarrow r_2 \\ \dots \\ d_n \rightarrow r_n \end{array}$$

where each d_i is a distinct name, and each r_i is a regular expression over the symbols in $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$, i.e., the basic symbols and the previously defined names.

- To distinguish name from symbols, the names are printed in **boldface** in regular definitions
- An **integer constant** can be defined as

digit → 0|1|2|3|4|5|6|7|8|9
int constant → **digit digit***

digit digit* (Starts with single digit and followed by zero or more additional digits).

Example : 528 (5 the first digit), (2 the second digit), (8 the third digit)

- An **identifier** as an initial letter followed by a sequence, possibly empty, of letters or digits can be defined as

letter → a|b|c|.....|z|A|B|.....|Z

digit → 0|1|2|.....|9

id → **letter (letter | digit)***

or

id → [a b cz A B C....Z] ([a b cz A B C....Z] [0 1 2...9])*

- **Unsigned numbers** (integer or floating point) are strings such as **5280**, **0.01234**, **6.336E4**, or **1.89E-4** can be defined as

digit	→	0 1 9
digits	→	digit digit*
optional_Fraction	→	. digits ε
optional_Exponent	→	(E (+ - e) digits) e
number	→	digits optional_Fraction optional_Exponent

- **optional_Fraction** is either a decimal point followed by one or more digits or empty string.

optional_Fraction → **. digits | ε**

(**'.'** fraction is present, **'.456'** fraction is present and consists of the digits,
ε fraction is absent)

optional_Exponent → **(E (+ | - | e) digits) | e**

(**E + 123**, **E-123**, **E**, **e**)

number → **digits optional_Fraction optional_Exponent**

123 (integer without fraction or exponent)

123.45 (integer with fraction, no exponent)

123e+45 (integer without fraction, with exponent)

0.123E-45 (fraction and exponent present)

456.789E (fraction present, exponent with an empty string)

Practice exercise

Write regular definitions for the following languages:

- All strings of lowercase letters that contain the five vowels in order.
- All strings of lowercase letters in which the letters are in ascending lexicographic order.
- Comments, consisting of a string surrounded by `/*` and `*/`, without an intervening `*/`, unless it is inside double-quotes `"`.
- All strings of a's and b's that do not contain the substring `abb`.
- All strings of a's and b's that do not contain the subsequence `abb`.

Solutions

a) All strings of lowercase letters that contain the five vowels (a, e, i, o, or u) in order.

Letter $\rightarrow$ [b-d f-h j-n p-t v-z]

String $\rightarrow$ (Letter|a)* (Letter|e)* (Letter|i)* (Letter|o)* (Letter|u)*

b) All strings of lowercase letters in which the letters in are in ascending lexicographic order.

String $\rightarrow$ a*b*c*d*.....z*

c) Comments, consisting of a string surrounded by /* and */, without an intervening */, unless it is inside double-quotes("")

Character $\rightarrow$ [a-zA-Z0-9]

Comment $\rightarrow$ /* (Character| "*" | ("*" | /) Character)* */

h) All strings of a's and b's that do not contain the substring abb.

$b^* (a (\in | b))^*$

i) All strings of a's and b's that do not contain the subsequence abb.

$b^* a^* (\in | b) a^*$

Extended Regular Expression

- **Kleene** introduced regular expressions with the basic operators for union, concatenation, and Kleene closure in the 1950s, many extensions have been added to regular expressions to enhance their ability to specify string patterns.

1. *One or more instances(+)*

- The unary postfix operator **+** represents the **positive closure of a regular expression and its language**.
- If ***r*** is a regular expression, the ***r*⁺** denotes the language **$(L(r))^+$**
- The operator **+** has the same precedence and associativity as the operator *****.
- Two useful algebraic laws,

$$r^* = r^+ | \epsilon$$

$$r^+ = rr^* = r^*r$$

relate the Kleene closure and positive closure.

2. Zero or one instance (?)

- The unary postfix operator **?** denotes **zero or one occurrence of the regular expression r** .
- The notation **$r^?$** is a shorthand for **$r \mid \epsilon$** .
- If **r** is a regular expression, then **$(r)^?$** is a regular expression that denotes the language
- The **?** operator has the same precedence and associativity as $*$ and $+$.

3. Character classes

- Consider a regular expression $a_1 \mid a_2 \mid \dots \mid a_n$.
- Now here each a_i is the symbol of the alphabet you can replace them by shorthand $[a_1, a_2, \dots, a_n]$ or by $[a_1 - a_n]$.
- We can describe identifiers as being strings generated by the regular expression, $[A-Za-z][A-Za-z0-9]^*$